Tasks: 気温予測アプリケーション

Input: Design documents from `/specs/001-open-meteo-forecast-app/`
Prerequisites: plan.md, spec.md, research.md, data-model.md, quickstart.md, contracts/open-meteo-api.yaml
Tests: ドライランとブラウザ確認で手動検証 (テストなし)
Organization: ユーザーストーリーごとにグループ化 (US1, US2, US3)

フォーマット: `[ID] [P?] [Story] Description`

- **ID**: 連番実行順 (例: T001, T002)
- **[P]**: 並列実行可能 (ファイル競合なし)
- **[Story]**: 実行ステップ (例: US1, US2, US3)
- フィルタリングされたタスクなしで、ユザストーリーのみ表示されます

パス規則

本プロジェクトは単一Webアプリのため単純。

- **フロントエンド**: `app/` (app.js, index.html, style.css)
- **ドキュメント**: `specs/001-open-meteo-forecast-app/` (plan.md, spec.md, research.md, data-model.md, quickstart.md, contracts/open-meteo-api.yaml)
- **テスト**: なし (ドライランとブラウザ確認で手動検証)

Phase 1: Setup (共通インフラ)

目的: プロジェクト初期化と基本構造の準備

- T001 [P] package.jsonにDev Dependencies追加確認 (Jest, Babel, Puppeteer 24.x, http-server 14.x)
- T002 [P] .gitignoreに*.log, coverage/, .DS_Storeを追加
- T003 [P] npm scripts追加確認 (test, test:watch, test:coverage, dev, lint)
- T004 ESLint設定ファイル(.eslintrc.json)作成し、ES6+コーディング規則

Checkpoint: 基盤完成 - ユーザーストーリー実装を並列開始可能

Phase 3: User Story 1 - 都市選択と気温表示 (Priority: P1) 🎯 MVP

ユーザーが世界中の主要都市を選択し、過去一年の天気データをグラフで確認する

Independent Test: ブラウザでUIが表示され、都市リストが表示されること確認、API通信でグラフ描画結果が期待値と一致することを確認

User Story 1 実装

- T011 [P] [US1] app/index.html にヘッダー(h1 id="cityName")とUI要素(div#loading, div#error, div#chart, div#cityInfo)を追加
- T012 [P] [US1] app/index.html にCanvas要素(id="temperatureChart", width="800", height="400")を追加
- T013 [P] [US1] app/app.js に定数定義(TimeOutMilliseconds, MaxRetries, Open-Meteo API URL)を追加
- T014 [US1] app/app.js に getCityFromURL() 関数を追加 (URLパラメータ解析)
- T015 [US1] app/app.js に updateUI(state) 関数を追加 (loading表示制御, エラーメッセージ表示, 都市情報表示)
- T016 [US1] app/app.js に fetchWithTimeout(url, timeoutMs) 関数を追加 (AbortSignal.timeout()使用, バックオフ対応)
- T017 [US1] app/app.js に fetchWithRetry(fetchFn, maxRetries) 関数を追加 (指数バックオフ: 1秒, 2秒, 4秒, 最大30秒)
- T018 [US1] app/app.js に processWeatherData(apiResponse) 関数を追加 (time, temperature_2m_max配列をdatapoints[{date, temperature}]に変換)
- T019 [US1] app/app.js に renderChart(datapoints, canvasId) 関数を追加 (Chart.js v4.4.0 Line Chart, 日付ラベル, 気温データ, 単位: 摂氏)
- T020 [US1] app/app.js に initialize() 関数を追加 (URLパラメータ解析, 初期UI状態設定, 都市データ取得, チャート描画, エラーハンドリング)

Checkpoint: ユーザーストーリー1が完全に動作し、独立検証可能

Phase 4: User Story 2 - 表示期間の調整 (Priority: P2)

Goal: ユーザーが過去一年分の表示期間を選択でき、より長期間または短期間のデータを確認できる

Independent Test: 期間選択ボタンをクリックし、グラフの表示範囲が変更されることを確認、API呼び出しとグラフ描画結果を確認

User Story 2 実装

- T021 [P] [US2] app/index.html に期間選択ボタンを追加 (id="1y", id="3y", id="5y", id="10y", id="custom")
- T022 [US2] app/app.js に getDateRangeFromPeriod(period) 関数を追加 (1y: 365日, 3y: 1095日, 5y: 1825日, 10y: 3650日)
- T023 [US2] app/app.js に updateURL(state) 関数を追加 (URLパラメータ更新, history.pushState()使用)
- T024 [US2] app/app.js に handlePeriodChange(period) 関数を追加 (period変更, データ再取得, チャート更新)
- T025 [US2] app/app.js に handleCustomDate(startDate, endDate) 関数を追加 (カスタム日付範囲, バリデーション)

Checkpoint: ユーザーストーリー1と2が両方独立に動作し、期間選択機能が正常に機能

Phase 5: User Story 3 - 温度単位の切り替え (Priority: P3)

Goal: ユーザーが摂氏(℃)華氏(℉)を切り替えて表示単位を変更できる

Independent Test: 単位スイッチをクリックし、グラフの温度表示と軸ラベルが変更されることを確認、localStorageに保存されることを確認

User Story 3 実装

- T031 [P] [US3] app/index.html に単位切り替ボタンを追加 (id="celsius", id="fahrenheit", id="unitToggle")
- T032 [US3] app/app.js に convertTemp(celsius, unit) 関数を追加 (℃: そのまま, ℉: celsius * 9/5 + 32, 小数点1桁)
- T033 [US3] app/app.js に getPreferredUnit() / setPreferredUnit(unit) 関数を追加 (localStorage.getItem('temperatureUnit'), localStorage.setItem('temperatureUnit', unit))
- T034 [US3] app/app.js に updateChartUnit(unit) 関数を追加 (Chart.js scale.ticks.callbackを更新, 軸ラベル変更, チャート再描画)
- T035 [US3] app/app.js に handleUnitChange(unit) 関数を追加 (単位変更, localStorage保存, チャート更新)

Checkpoint: ユーザーストーリー1,2,3が独立に動作し、期間選択と単位変更が相互に干渉しない

Phase 6: Polish & Cross-Cutting Concerns

目的: 複数ユーザーストーリーに影響する改善

- T041 [P] README.mdの更新 (日本語スペック, セットアップ手順, スクリーンショット追加)
- T042 [P] app/app.js にJSDoc形式コメント追加 (関数名・パラメータ・戻り値の説明)
- T043 [P] app/app.js にconsole.log()デバッグ出力 (リリース時はlogLevel変数でON/OFF切り替え)
- T044 [P] app/app.js にconsole.error()エラーログ出力 (ユーザーフレンドリーなエラーメッセージと併用)
- T045 [P] ESLint実行 (npm run lint) 警告ゼロ確認、コードスタイル統一
- T046 [P] Chrome DevTools Performance タブでパフォーマンス確認 (初期ロード3秒以内, チャート描画1秒以内, メモリリーク確認)
- T047 [P] Chrome DevTools Network タブでAPIリクエスト確認 (URL正しさ, パラメータ正しさ, キャッシュ動作)
- T048 [P] Chrome DevTools Console タブでエラーなし確認 (ブラウザ互換性問題なし)
- T049 [P] Firefox, Safari, Edge で動作確認 (クロスブラウザ互換性テスト)
- T050 [P] GitHub Pages デプロイ設定 (main ブランチ / root ディレクトリ)
- T051 [P] 本番環境でHTTPS動作確認 (GitHub Pages デフォルトHTTPS)
- T052 [P] 本番環境でCORSエラーなし確認 (Open-Meteo API は CORS 対応済み)
- T053 [P] 本番環境でAPIレート制限テスト (10,000リクエスト/日 以内)
- T054 [P] 本番環境でlocalStorage動作確認 (プライベートブラウズモード, ストレージ容量制限)
- T055 [P] 本番環境でURLパラメータ動作確認 (ブックマーク, 共有, ブラウザ戻る/進む)
- T056 [P] 本番環境でモバイルブラウザ確認 (iOS Safari, Chrome for Android)
- T057 [P] 本番環境でアクセシビリティ確認 (キーボードナビゲーション, スクリーンリーダー, ARIA属性)
- T058 [P] 本番環境でエラーシナリオ確認 (ネットワークオフライン, 無効な都市名, 空データ, 長期間選択)
- T059 [P] 本番環境でパフォーマンス最適化確認 (Lighthouse スコア 90+)
- T060 [P] 本番環境でセキュリティ確認 (XSS, CSRF, Content Security Policy)

Checkpoint: 全ユーザーストーリー完了、本番環境で安定動作

Dependencies & Execution Order

フェーズ依存関係

- **Setup (Phase 1)**: 依存なし - 即座に開始可能
- **Foundational (Phase 2)**: Setup完了後 - ブロック: ユーザーストーリー全ブロック
- **User Stories (Phase 3-5)**: Foundational完了後に開始可能
  - ユーザーストーリーは並列実行可能 (異なるメンバー担当)
  - または優先順位に従って順次実行 (P1 → P2 → P3)
- **Polish (Phase 6)**: すべてのユーザーストーリー完了後

ユーザーストーリー依存関係

- User Story 1 (P1): Foundational完了後に開始可能 - 他ストーリーへの依存なし (MVP)
- User Story 2 (P2): Foundational完了後に開始可能 - US1の実装を拡張するが独立テスト可能
- User Story 3 (P3): Foundational完了後に開始可能 - US1/US2に依存するが独立テスト可能

ユーザーストーリー内の実装順

- HTML変更最初 → JavaScript実装 (DOM要素が必要なため)
- 初期設定後 → イベントリスナー実装 (関数が存在するため)
- データ処理 → グラフ描画 (データが必要なため)
- 基本実装 → エラーハンドリング追加 (基本が動いてから)

並列実行機会

- Phase 1: T002, T003, T004 [P] (異なるタスク)
- Phase 2: T007, T008, T009, T010 [P] (異なるモジュール)
- Phase 3: T011, T012, T013 [P] (異なるHTML要素)
- Phase 4: T021, T022, T023 [P] (異なるUI要素)
- Phase 5: T031, T032, T033, T034 [P] (異なるUI要素)
- Phase 6: T039, T040, T041, T042, T043 [P] (異なるファイル)

Parallel Example: User Story 1

```
# User Story 1の並列実行例:
Task: "T011 [P] [US1] app/index.html にヘッダーとUI要素を追加"
Task: "T012 [P] [US1] app/index.html にCanvas要素を追加"
Task: "T013 [P] [US1] app/app.js に定数定義を追加"

Task: "T014 [US1] fetchWeatherData()関数実装"
Task: "T015 [US1] updateUI()関数実装"
Task: "T016 [US1] formatWeatherDataForChart()を実行(依存あり)"

Task: "T017 [US1] チャートインスタンス生成"
Task: "T018 [US1] ウィンドウリサイズ対応"
Task: "T019 [US1] エラーハンドリング追加"
```

Implementation Strategy

MVP First (User Story 1のみ)

1. Phase 1: Setup完了
2. Phase 2: Foundational完了 (CRITICAL - すべてのストーリーをブロック)
3. Phase 3: User Story 1完了
4. **ストップ&バリデート**: User Story 1を独立テスト
5. デプロイ/デモ準備完了

段階的デリバリー

1. Setup + Foundational → 基盤完成
2. User Story 1追加 → 独立テスト → デプロイ/デモ (MVP!)
3. User Story 2追加 → 独立テスト → デプロイ/デモ
4. User Story 3追加 → 独立テスト → デプロイ/デモ
5. 各ストーリーが前のストーリーを壊さず価値追加

並列チーム戦略

複数開発者がいる場合:

1. チーム全員: Setup + Foundational完了
2. Foundational完了後:
   - 開発者A: User Story 1 (MVP)
   - 開発者B: User Story 2 (期間調整)
   - 開発者C: User Story 3 (単位切り替え)
3. 各ストーリーが独立にテスト・統合可能

独立テスト戦略

- **US1**: 都市選択 → グラフ表示 → エラーハンドリング
- **US2**: 期間選択 → データ再取得 → グラフ更新
- **US3**: 単位切り替え → 変換計算 → 軸ラベル更新

推奨MVPの理由

最小限のUser Story 1 (P1) 実装だけで十分:

- 都市選択機能 (10都市)
- Open-Meteo API連携
- Chart.jsでの描画 (過去1年のデータ)
- エラーハンドリング

これだけで世界の都市の気温を確認できます。プロトタイプ段階最適の形。

Notes

- **[P]**: 並列実行 - 異なるファイル、依存関係なし
- **[Story]**: タスクを特定のユーザーストーリーに紐付け、トレーサビリティ確保
- **独立実装**: 各ストーリーは他ストーリーに依存せずに実装・テスト可能
- **TDD**: 明確要求: テスト先行で実装前に失敗確認 (テストなしの場合、アドホックテスト推奨)
- **コミット**: 各タスク完了後または論理的グループ完了ごとにコミット推奨
- **チェックポイント**: 各ストーリー完了後に独立検証実施
- **並列実行**: 異なるファイル、同一ファイル編集なし、ユーザー独立性確保の原則適用

Version: 1.0.0
Created: 2025-11-25
Generated by: speckit.tasks
Based on: plan.md (Phase 0-5), spec.md (User Stories), data-model.md, research.md, contracts/open-meteo-api.yaml